

Basic Linux Commands

1. **ls** it's used to display a list of files and sub-directories in the current directory

Options Description

ls -a	list all files including hidden file starting with '.'.
ls -d	list directories - with ' */'.
ls -l	list with long format - show permissions.
ls -F	Append indicator (one of */=>@) to entries.
ls -lh	This command will show you the file sizes in human readable format.
ls -r	list in reverse order.
ls -i	list file's inode(index) number.
ls -ltr	View Reverse Output Order by Date.
ls -t	sort by time & date.
ls -n	It is used to print group ID and owner ID instead of their names.
ls -m	A list of entries separated by commas should fill the width.
ls -g	This allows you to exclude the owner and group information columns.
ls -q	Force printing of non-graphic characters in file names as the character '?';.
ls -Q	Place double quotations around the entry names.

2. **Man-** the manual pages of the command.

3. **Cd** Change directory

4. **Echo** file name ---- to create the file

5. `Cat > file name ---` to write the contents , press `ctrl+d` to get out

6. Touch- to create a file and change the timestamp of file with various options

Can create more than one file

`Touch file1 file 2`

Can create array of files

`Touch filenew{1..10}`

`Touch filenew{a..e}`

7. `mkdir & rmdir` — Use the `mkdir` command when you need to create a folder or a directory.

For example, if you want to make a directory called “DIY”, then you can type `“mkdir DIY”`. Remember, as told before, if you want to create a directory named “DIY Hacking”, then you can type `“mkdir DIY\ Hacking”`.

8. Use `rmdir` to delete a directory. But `rmdir` can only be used to delete an empty directory.

To delete a directory containing files, use `rm`.

9. rm - Use the **rm** command to delete files and directories. Use "**rm -r**" to delete just the directory. It deletes both the folder and the files it contains when using only the **rm** command.

10. cp — Use the **cp** command to copy files through the command line.

It takes two arguments:

The first is the location of the file to be copied, the second is where to copy.

11. mv — Use the **mv** command to move files through the command line.

We can also use the **mv** command to rename a file.

For example, if we want to rename the file "**text**" to "**new**", we can use "**mv text new**".

It takes the two arguments, just like the **cp** command.

12. grep: The 'grep' command stands for "**global regular expression print**". grep command filters the content of a file which makes our search easy.

It is a command-line utility to search plain-text data groups for lines that are the same as a regular expression. The name "**grep**" comes from the command, i.e., **ed**, which contains the same effect. Originally, grep was designed for the Unix operating system, but it became available for every Unix-like system later and a few others like OS 9.

The grep filter finds a file for a specific character pattern and shows every line that includes that pattern. Characters should be in quotation marks if they occur in the pattern parameter with a special meaning for the shell. Usually, we must enclose the whole pattern in one quotation mark if the pattern parameter is not a common string.

In an expression like **[a-z]**, the minus sign (-) cml describes a range based on the current collating order. A collating order may specify equivalent classes in character ranges for use. The grep command assumes stdin when no files are mentioned.

The pattern is called the regular expression that is found inside the file. The pattern is restricted regular expressions in the format of the **egrep** or **ed** command. The grep

command applies a solid non-deterministic algorithm. It comes in handy when we need to filter large log files.

How to install grep in Linux?

Grep comes pre-installed in almost every distribution of Linux. However, in case, we can install it with the below command in the terminal window if it is missing from our system:

1. `$ sudo apt-get install grep`

grep with pipe

1. `command | grep <searchWord>`

Example:

1. `cat marks.txt | grep 9`

grep without pipe

It can be used without pipe also.

Syntax:

1. `grep <searchWord> <file name>`

Example:

1. `grep 9 marks.txt`

grep options

- **grep -vM:** The 'grep -v' command displays lines not matching to the specified word.

Syntax:

1. `grep -v <searchWord> <fileName>`

Example:

1. `grep -v 9 marks.txt`

13.

cat >file	Places the standard input into the file
more file	Output the contents of the file
head file	Output the first 10 lines of the file
tail file	Output the last 10 lines of the file

13. FIND : The find command is used to search for and [locate a list of files and directories](#) based on the conditions you specify, matching the arguments.

Part I – Basic Find Commands for Finding Files with Names

When it comes to finding files with specific names, the find command offers a range of options to streamline the process. Here are some basic find commands for locating files based on their names.

1. Find Files Using Name in Current Directory

Find all the files whose name is **tecmint.txt** in a current working directory.

```
# find . -name tecmint.txt

./tecmint.txt
```

Find all the files under **/home** directory with the name **tecmint.txt**.

```
# find /home -name tecmint.txt

/home/tecmint.txt
```

4. Find Directories Using Name

Find all directories whose name is **Tecmint** in **/** directory.

```
# find / -type d -name Tecmint  
  
/Tecmint
```

6. Find all PHP Files in the Directory

Find all **php** files in a directory.

```
# find . -type f -name "*.php"
```

Find all Empty Files

To find all empty files under a certain path.

```
# find /tmp -type f -empty
```

Find all Empty Directories

To file all empty directories under a certain path.

```
# find /tmp -type d -empty
```

14. IFCONFIG : The `ifconfig` command prints an output when executed without arguments or with the interface name only. If used with arguments, there is no output.

Variation in Ifconfig command:

a)

Option	Description
<code>-a</code>	Prints the configuration for all interfaces, not just the active ones.
<code>-s</code>	Displays only a shortlist of all interfaces.
<code>-v</code>	Prints a more detailed configuration for all interfaces.
<code>interface_name</code>	Represents the interface's name and is accompanied by a unit number. Prints the configuration for a specific interface.
<code>down</code>	Deactivates the interface.
<code>up</code>	Activates the interface.
<code>[IP_address_number]</code>	Assigns an IP address to a specified interface.

HOW TO ENTER IN ROOT DIRECTORY INSTEAD OF CURRENT

--: USING `sudo -i`

```
ubuntu@ubuntu:~$ pwd
/home/ubuntu
ubuntu@ubuntu:~$ sudo -i
root@ubuntu:~# whoami
root
root@ubuntu:~# pwd
/root
root@ubuntu:~#
```

Root password changed from

Sudo passwd root

(abc@123 given)

>>>> To add new user

```
root@ubuntu:~# sudo adduser user1
Adding user `user1' ...
Adding new group `user1' (1000) ...
Adding new user `user1' (1000) with group `user1' ...
Creating home directory `/home/user1' ...
Copying files from `/etc/skel' ...
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
Changing the user information for user1
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
root@ubuntu:~#
```

>> To add group

```
root@ubuntu:~# addgroup g1
Adding group `g1' (GID 1002) ...
Done.
```

Chown command:

The main command for changing ownership is chown. It allows users to change user and group ownership both

for files and for directories. We'll go over the chown syntax below:

chown [OPTION] OWNER[:GROUP] FILE(s)

e.g

chown username filename

chown user1 file1.txt

Chgrp to change group ownership

```
root@ubuntu:~# touch aa{1..5}
root@ubuntu:~# ls -l
total 0
-rw-r--r-- 1 root root 0 Aug 22 06:02 aa1
-rw-r--r-- 1 root root 0 Aug 22 06:02 aa2
-rw-r--r-- 1 root root 0 Aug 22 06:02 aa3
-rw-r--r-- 1 root root 0 Aug 22 06:02 aa4
-rw-r--r-- 1 root root 0 Aug 22 06:02 aa5
root@ubuntu:~# chown u1 aa1
root@ubuntu:~# chown g1 aa5
chown: invalid user: 'g1'
root@ubuntu:~# chgrp g1 aa5
```

To enter in particular user from root

```
root@ubuntu:~# su -l u1
```

